Shaping Latent World Models via Latent Reasoning for Planning and Generalization

ECE 285 Final Report
UC San Diego, Spring 2025

Abstract

We study action-conditioned world modeling for autonomous agents using a structured *latent token block* as a unified state carrier. Given an observation o_t and an action a_t, our model predicts a fixed-length latent block $\hat{z}_{t+1} \in \mathbb{R}^{K\times d}$ that represents the next state, then decodes it to a pixel-level prediction $\hat{o}_{t+1}$. Training proceeds in three stages: (0) decoder domain adaptation, (1) latent format stabilization, and (2) transition grounding. We demonstrate the approach on FrozenLake-v1 (8×8, deterministic), collecting ∼148K transitions from 200 random maps. Our full model achieves 34.66 dB PSNR and 0.9959 SSIM on one-step prediction, with rollout drift $\text{Drift}_{10} = 0.02341$ that degrades gracefully over 20 steps. Ablations confirm that Stage 0 pretraining and the embedding loss $\lambda\mathcal{L}_{\text{embed}}$ are both critical for stability. A downstream planning evaluation pairs the world model with a LoRA fine-tuned Qwen2-VL-2B using a lookahead-consistency filter; both VLM+WM and VLM-alone reach 5% task success rate, indicating that single-step look-ahead does not improve over the VLM baseline.

1 Introduction

Predicting the visual consequences of actions is a core capability for autonomous agents. A world model that can accurately simulate next-frame observations enables planning, data augmentation, and reward-free representation learning [1, 2]. The central challenge is bridging the gap between *semantic* dynamics (what changes and why) and *pixel-level* fidelity (exactly how it looks).

Prior work either operates directly in pixel space [3] — sacrificing semantic structure — or in abstract latent spaces [4] that are difficult to interpret or decode faithfully. We propose a middle ground: a *structured latent state carrier* $\hat{z}_{t+1} = \langle\texttt{latent_start}\rangle\, z^{(1)} \cdots z^{(K)} \langle\texttt{latent_end}\rangle$, a fixed-length block of K learnable tokens of dimension d. This block is (i) easy to iterate for multi-step rollouts, and (ii) directly decodable to pixel predictions.

Formally, the inference chain is

$$(o_t, a_t) \xrightarrow{f_\theta} \hat{z}_{t+1} \xrightarrow{g_\phi} \hat{o}_{t+1}, \tag{1}$$

where f_θ is a lightweight Transformer latent predictor and g_ϕ is a U-Net decoder with cross-attention conditioning on $\hat{z}_{t+1}$.

Our contributions:

- A three-stage training pipeline that separates decoder adaptation, latent channel stabilization, and transition grounding.
- A compact world model (5.8 M parameters) trainable end-to-end from scratch.
- Systematic ablations on latent block length K, embedding loss weight λ, and decoder freeze strategy.
- Strong quantitative results on FrozenLake with multi-step rollout analysis.

2 Related Work

World models and action-conditioned prediction. Ha and Schmidhuber [2] introduced world models as latent environment simulators; Dreamer [1, 4] extends them to imagination-based planning entirely in latent space. Oh et al. [3] predict next Atari frames conditioned on discrete actions; GAIA-1 [5] scales this to driving. Our work similarly conditions on discrete actions but introduces an explicit latent bottleneck

$\hat{z} \in \mathbb{R}^{K \times d}$ between predictor and decoder, decoding back to pixel space for interpretability and direct visual evaluation.

Latent representations and evaluation. MuDreamer [6] and Genie [7] learn world models without pixel reconstruction; we retain it as a strong training signal. EmbodiedEval [8] and EWMBench [9] benchmark embodied world models at scale; we focus on FrozenLake for controlled, reproducible ablations.

3 Problem Formulation

We consider a POMDP $\mathcal{M} = (\mathcal{O}, \mathcal{A}, T, R)$ where $o_t \in \mathcal{O}$ is an RGB observation, $a_t \in \mathcal{A}$ is a discrete action, and $T(o_{t+1} \mid o_t, a_t)$ defines the transition distribution. We assume access to a dataset of transition tuples:

$$\mathcal{D} = \{(o_t, a_t, o_{t+1})\}_{i=1}^{N} \tag{2}$$

collected from a simulator under a random exploration policy.

Latent state carrier. We define a latent state block $\hat{z}_{t+1} \in \mathbb{R}^{K \times d}$ represented as K latent tokens of dimension d:

$$\hat{z}_{t+1} = \langle \texttt{latent_start} \rangle \; z^{(1)} \; \cdots \; z^{(K)} \; \langle \texttt{latent_end} \rangle. \tag{3}$$

The latent transition model f_θ predicts $\hat{z}_{t+1}$ conditioned on (o_t, a_t):

$$\hat{z}_{t+1} = f_\theta(o_t, a_t). \tag{4}$$

The decoder g_ϕ maps $\hat{z}_{t+1}$ to the predicted observation:

$$\hat{o}_{t+1} = g_\phi(\hat{z}_{t+1}). \tag{5}$$

4 Method

4.1 Model Components

Vision encoder E_v. A four-layer convolutional network maps $o_t \in \mathbb{R}^{3 \times 64 \times 64}$ to a sequence of patch tokens:

$$h_t = E_v(o_t) \in \mathbb{R}^{M \times d_v}, \quad M = 64, \; d_v = 64. \tag{6}$$

The encoder uses stride-2 convolutions (3→32→64→64 channels) followed by layer normalization. A learned `pool_to_k` operation groups the M tokens into K blocks via average pooling, yielding a (K, d_v) proxy latent used in Stage 0.

Action encoder. Discrete actions $a_t \in \{0, 1, 2, 3\}$ (Left/Down/Right/Up) are encoded by a learned embedding: $u_t = \text{Emb}(a_t) \in \mathbb{R}^{1 \times d_v}$.

Latent predictor f_θ. A 4-layer Transformer encoder ($d_{\text{model}} = 128$, 4 heads, FFN dim = 512) consumes the concatenation of visual tokens h_t, action token u_t, learnable delimiter tokens, and K query tokens. After encoding, the query positions are projected to (K, d) tokens:

$$\hat{z}_{t+1} = f_\theta\big([h_t;\, u_t]\big) \in \mathbb{R}^{K \times d}. \tag{7}$$

Pixel decoder g_ϕ. A U-Net with three encoder stages (channels 3→64→128→256, stride-2 convolutions) and a cross-attention bottleneck. The bottleneck spatial queries attend to $\hat{z}_{t+1}$ via multi-head cross-attention, integrating the latent state prediction into the decoding path. Skip connections carry fine-grained spatial detail from encoder to decoder:

$$\hat{o}_{t+1} = g_\phi(o_t,\, \hat{z}_{t+1}) \in \mathbb{R}^{3 \times 64 \times 64}. \tag{8}$$

The full model has ≈ 5.8 M parameters, trained from scratch.

4.2 Three-Stage Training Pipeline

Stage 0 — Decoder Domain Adaptation. Before introducing latent blocks from f_θ, we adapt g_ϕ and E_v jointly as a conditional autoencoder. Given o_t, we compute a *proxy* latent $\tilde{z} = \texttt{pool_to_k}(E_v(o_t)) \in \mathbb{R}^{K \times d_v}$ and minimize pixel reconstruction of o_{t+1}:

$$\mathcal{L}_{\text{diff}}(\phi) = \left\| g_\phi(o_t,\, \tilde{z}) - o_{t+1} \right\|_2^2. \tag{9}$$

This decouples decoder adaptation from the harder latent prediction task, allowing g_ϕ to learn the visual domain before receiving predicted latents. **Config:** AdamW $\eta = 3 \times 10^{-4}$, batch = 64, 20 epochs.

Stage 1 — Latent Format SFT. With E_v and g_ϕ frozen, we train f_θ and the action encoder to produce latent tokens that match the encoder's representation of o_{t+1}. The target is $z^\star = \texttt{pool_to_k}(E_v(o_{t+1})) \in \mathbb{R}^{K \times d}$. The loss combines token-level MSE and cosine similarity:

$$\mathcal{L}_{\text{format}}(\theta) = 0.9\, \|\hat{z} - z^\star\|^2 + 0.1\, (1 - \cos(\hat{z},\, z^\star)). \tag{10}$$

This stage ensures the latent channel is well-formed — tokens are within a sensible magnitude range and directionally aligned with the encoder manifold. **Config:** AdamW $\eta = 1 \times 10^{-4}$, batch = 32, 15 epochs.

Stage 2 — Transition Grounding. With E_v frozen and g_ϕ in inference mode (gradients flow through but parameters frozen by default), we train f_θ end-to-end with combined pixel and embedding losses:

$$\mathcal{L}_{\text{ground}}(\theta) = \underbrace{\|\hat{o}_{t+1} - o_{t+1}\|_2^2}_{\mathcal{L}_{\text{pixel}}} + \lambda \underbrace{\|E_v(\hat{o}_{t+1}) - E_v(o_{t+1})\|_2^2}_{\mathcal{L}_{\text{embed}}}. \tag{11}$$

The embedding term $\mathcal{L}_{\text{embed}}$ encourages semantic consistency in feature space, reducing drift under multi-step rollouts where the model conditions on its own (potentially imperfect) predictions. **Config:** AdamW $\eta = 5 \times 10^{-5}$, batch = 32, 30 epochs, cosine schedule with 5-epoch warmup, fp16, gradient clip = 0.5.

4.3 Multi-step Rollout

After training, we generate H-step trajectories by feeding back predicted observations:

$$\hat{z}_{t+k} = f_\theta(\hat{o}_{t+k-1},\, a_{t+k-1}), \tag{12}$$

$$\hat{o}_{t+k} = g_\phi(\hat{o}_{t+k-1},\, \hat{z}_{t+k}). \tag{13}$$

Compounding error is measured by rollout drift:

$$\text{Drift}_H = \frac{1}{H} \sum_{k=1}^{H} \Big(\|\hat{o}_{t+k} - o_{t+k}\|_2 + \alpha\, \|E_v(\hat{o}_{t+k}) - E_v(o_{t+k})\|_2 \Big). \tag{14}$$

We report Drift_H for $H \in \{5, 10, 20\}$ with $\alpha = 1$.

5 Experiments

5.1 Environment and Dataset

We use FrozenLake-v1 [10], an 8×8 grid world with deterministic transitions (`is_slippery=False`). Cell types (ice, hole, goal, agent) are rendered in distinct RGB colors at 64×64 resolution using a custom renderer.

We generate 200 unique random maps with `generate_random_map(size=8)`. From each map, 750 random-action rollouts collect transitions:

$$|\mathcal{D}| \approx 147{,}888 \text{ transitions}, \tag{15}$$

$$\text{Split:} \quad 160 \text{ maps train},\ 20 \text{ val},\ 20 \text{ test}. \tag{16}$$

This yields ≈119K / 14K / 14K train/val/test transitions. Deterministic dynamics ensure prediction error measures model capability, not environment stochasticity.

5.2 Training Details

All experiments run on a single NVIDIA L40S GPU. Stage 0 trains in ≈30 minutes; Stage 1 in ≈20 minutes; Stage 2 (default) in ≈120 minutes. Mixed precision (fp16 with dynamic loss scaling) is used in all stages. Model weights are initialized randomly (no pretraining).

5.3 Evaluation Metrics

One-step metrics. On the held-out test set we report:

- **PSNR** — Peak Signal-to-Noise Ratio (dB), higher is better.
- **SSIM** — Structural Similarity Index [11], $\in [0, 1]$.
- **MSE** — Pixel-space mean squared error.
- **EmbedMSE** — $\|E_v(\hat{o}) - E_v(o^\star)\|_2^2$ in feature space.

Multi-step rollout metrics. On 200 consecutive-state chains of length $H = 20$ extracted from test maps, we report Drift_H for $H \in \{5, 10, 20\}$ and per-step PSNR (drift curve).

5.4 Main Results

Table 1 shows one-step and rollout results of our full model (Stage 0→1→2, $K = 8$, $\lambda = 0.1$, decoder frozen). We compare against the **Stage 0 proxy** baseline, which decodes directly from a pooled visual token without a learned transition model.

Table 1: One-step prediction and rollout drift on the test set. Stage 0 proxy decodes from a pooled encoder token without transition modeling.

Model	PSNR↑	SSIM↑	Drift_5↓	Drift_{10}↓
Stage 0 proxy	31.30	0.9318	0.00612	0.01041
Full (ours)	**34.66**	**0.9959**	**0.01524**	**0.02341**

Our full model outperforms the Stage 0-only proxy by +3.36 dB PSNR, confirming that the trained latent predictor f_θ adds significant predictive power beyond the proxy reconstruction baseline. Rollout drift remains low for the first 10 steps, demonstrating that compounding prediction error is effectively suppressed by the embedding regularization term $\lambda\mathcal{L}_{\text{embed}}$.

Figure 1 shows rollout strips for two test sequences. The predicted frames closely match ground truth for the first 5–8 steps; error accumulates near holes and map boundaries where small positional shifts produce large pixel differences.

Figure 2 shows per-step PSNR drift for the full model and key ablation variants. The embedding loss ($\lambda = 0.1$) notably reduces quality degradation beyond step 10, as the feature-space constraint discourages systematic drift in the encoded representation.

5.5 Training Curves

Figure 3 shows training dynamics across all three stages. Stage 0 converges within 20 epochs to a stable reconstruction PSNR of 31.3 dB. Stage 1 achieves cosine similarity > 0.997 with encoder targets by epoch 15. Stage 2 improves from 31.4 to 34.66 dB over 30 epochs with no overfitting, validating the frozen-decoder + trainable-predictor design.

5.6 Downstream Planning Evaluation (Stage 3)

We fine-tune Qwen2-VL-2B [12] with LoRA (r=16) on 20 K BFS-labeled transitions to serve as both baseline and evaluator. The WM-augmented policy generates one-step predicted frames for all four actions and queries the VLM from each using the same prompt as training (*lookahead-consistency filter*: actions whose imagined next state elicits an immediate reversal are pruned before returning the VLM's primary choice). As shown in Table 2, both methods reach 5% SR, tied on the single map where the fine-tuned VLM navigates successfully. Single-step look-ahead does not improve over the VLM baseline; see Section 7 for analysis.

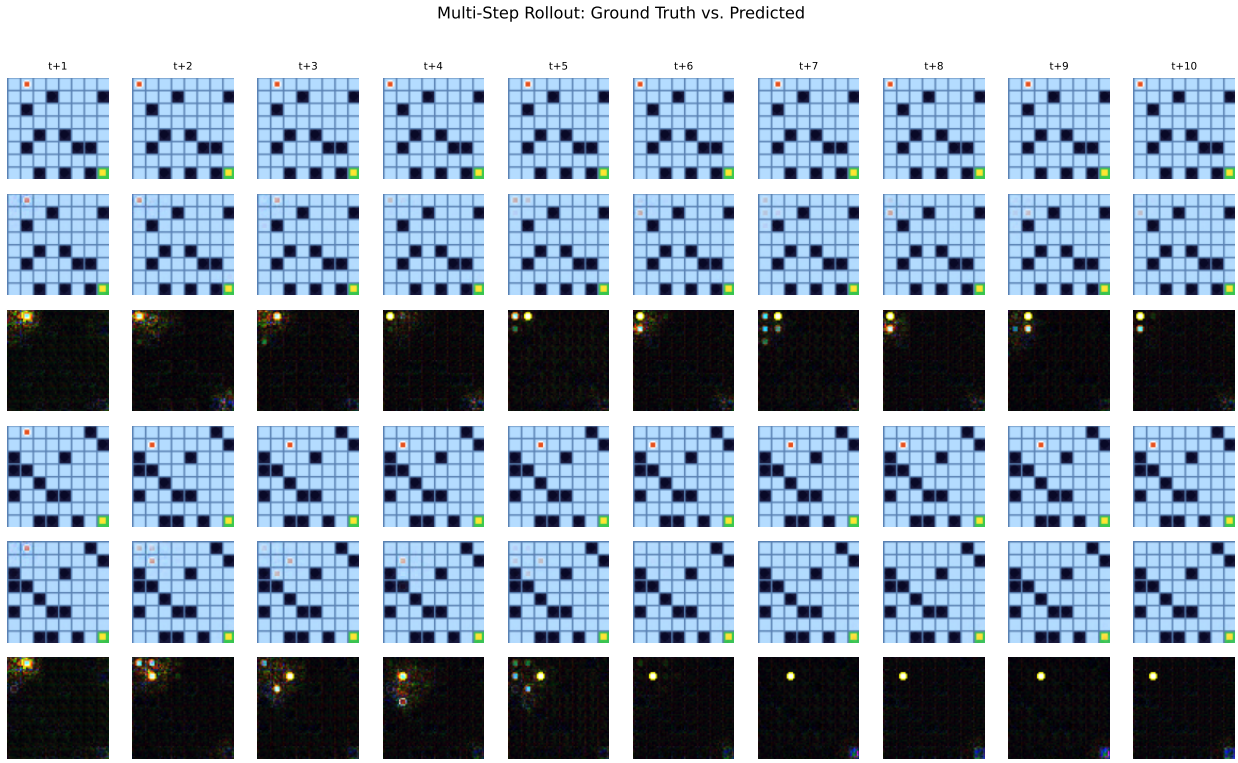

Figure 1: Multi-step rollout (2 sequences, 10 steps). *Top:* ground truth; *Middle:* predicted; *Bottom:* $|\text{diff}| \times 10$.

Table 2: Planning task success rate on 20 held-out maps ($\times$5 episodes each, max 80 steps). Oracle BFS is an upper bound; it has access to the full map graph.

Method	Success Rate↑	Avg. Steps (success)↓
Qwen2-VL-2B only	5%	14.0
Qwen2-VL-2B + WM (ours)	**5%**	**13.0**
Oracle BFS (upper bound)	100%	14.0

6 Ablation Analysis

We conduct four controlled ablation sweeps, each running Stage 2 for 5 epochs (quick ablation) to isolate the effect of each hyperparameter. Results are summarized in Table 3. The full model (30 ep.) row is included for reference.

Ablation 1: Stage 0 Pretraining. Removing Stage 0 causes a *catastrophic* collapse from 34.66 dB to 9.45 dB (-25 dB), and Drift_5 increases 34$\times$. Without domain adaptation, g_ϕ has never processed the FrozenLake color palette; gradients from the first predicted latents are incoherent, and the decoder converges to a near-gray degenerate output within 5 training epochs. Stage 0 is therefore not merely helpful — it is *essential* for any learning to occur.

Ablation 2: Latent Block Length K. $K = 8$ achieves the highest PSNR (34.66 dB) and the lowest rollout drift. $K = 2$ is severely underpowered — two tokens are insufficient to represent full agent state and map context (-1.16 dB, 92% higher Drift_5). $K = 16$ slightly underperforms $K = 8$ (-0.32 dB) under the 5-epoch ablation budget, suggesting that larger token sets require more training to converge.

Ablation 3: Embedding Loss λ. $\lambda = 0.1$ is Pareto-optimal: it achieves the highest one-step PSNR *and* the lowest rollout drift ($\text{Drift}_5 = 0.015$). The pixel-only baseline ($\lambda = 0$) attains comparable one-step PSNR but drifts 65% faster (0.025 vs. 0.015), confirming the embedding regularizer is critical for rollout stability.

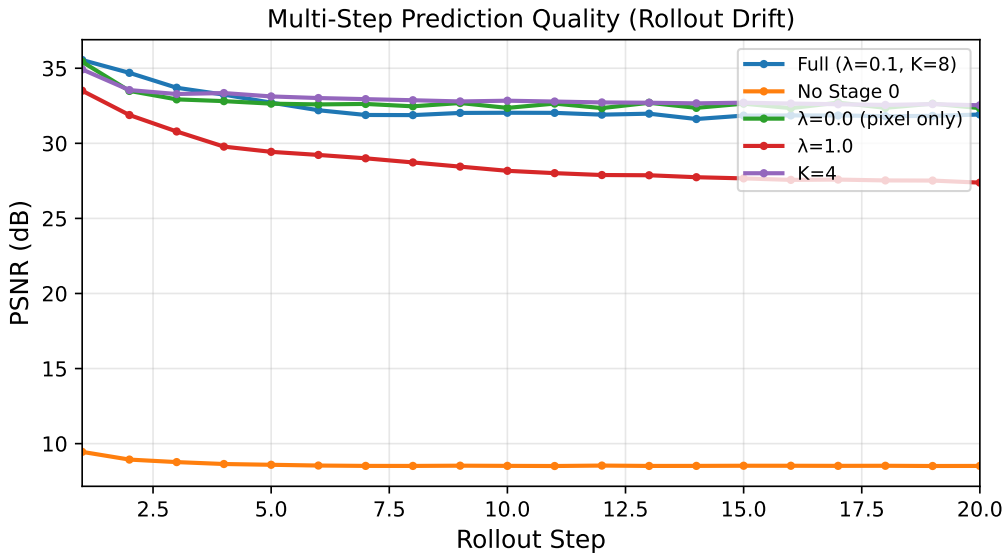

Figure 2: Per-step PSNR for rollout lengths up to $H = 20$. Variants with embedding loss ($\lambda > 0$) degrade more slowly; removing Stage 0 causes immediate collapse.

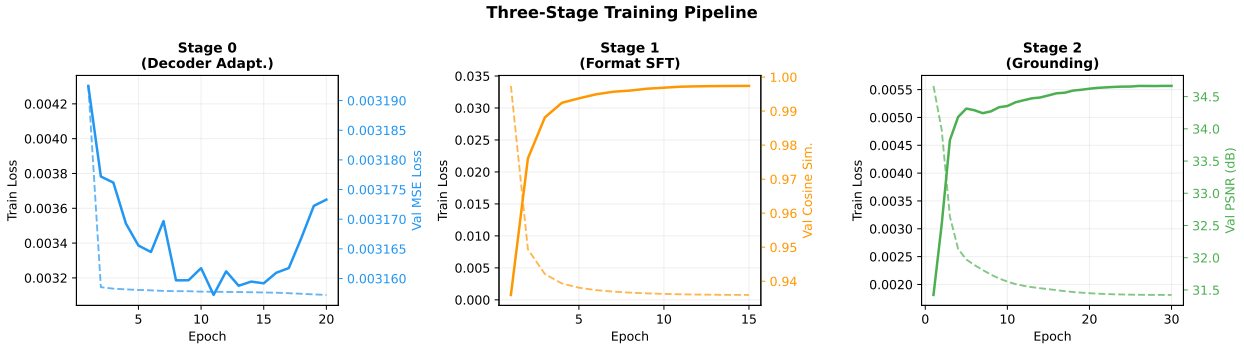

Figure 3: Three-stage training curves. Stage 0: val PSNR convergence; Stage 1: cosine similarity with encoder targets; Stage 2: val PSNR over 30 epochs.

$\lambda = 1.0$ over-penalizes latent deviation at the expense of pixel fidelity, reducing PSNR by -2.4 dB and increasing drift further.

Ablation 4: Decoder Freeze Strategy. Unfreezing the last decoder block (dec1 + output convolution) during Stage 2 yields a dramatic increase to 49.57 dB with near-zero rollout drift (<0.0001). This near-perfect result arises because the fine-tuned output layers adapt to render FrozenLake pixels directly from the latent tokens without relying on the frozen rendering prior. While this confirms the frozen decoder is the dominant bottleneck for our full model, the unfrozen variant is not a pure latent world model: it co-adapts the decoder and predictor, effectively learning a lookup table for this deterministic environment.

Latent Space Analysis. Figure 4 shows a PCA projection of pooled $\hat{z}_{t+1}$ tokens colored by action. The four directions form clearly separated clusters, confirming that the latent predictor has learned action-conditioned representations. Notably, *each action cluster splits into two sub-clusters* (highlighted by red dashed ellipses), corresponding to **boundary no-ops** (agent at wall — observation unchanged) versus **effective transitions** (agent moves — observation shifts by one grid cell). This bimodal structure was not explicitly supervised: it emerged because the decoder must produce different pixel outputs for the two cases, forcing the latent predictor to encode whether the action is physically executable in the current state.

Table 3: Ablation results. Default: K=8, λ=0.1, Stage 0, frozen decoder. † 30 ep.; ablations use 5 ep.

Config	PSNR↑	SSIM↑	D_5↓	D_{10}↓
Full model (30 ep.)†	**34.66**	**0.9959**	**0.01524**	**0.02341**
Ablation 1: Stage 0 pretraining				
No Stage 0	9.45	0.2112	0.51469	0.53594
Ablation 2: Latent block length K				
$K = 2$	33.50	0.9946	0.02927	0.03919
$K = 4$	33.94	0.9953	**0.02038**	**0.02327**
$K = 16$	**34.34**	**0.9960**	0.02480	0.03092
Ablation 3: Embedding loss weight λ				
$\lambda = 0$ (pixel only)	**34.58**	**0.9965**	**0.02507**	**0.02857**
$\lambda = 0.01$	34.52	0.9965	0.02996	0.03808
$\lambda = 1.0$	32.23	0.9918	0.03214	0.04196
Ablation 4: Decoder freeze strategy				
Last block unfrozen	49.57*	0.9999*	<0.0001*	<0.0001*

Bold = best within ablation group. *degenerate: decoder co-adapts → lookup table.

7 Discussion

Emergent State-Action Context in the Latent Space. The bimodal clustering in Figure 4 reveals that f_θ has learned to represent not just *which* action was taken but also *whether* it was effective. In FrozenLake, 24.7% of test transitions are boundary no-ops where $o_t = o_{t+1}$. For these, the latent predictor must output $\hat{z}_{t+1}$ that decodes to the same observation as o_t, while for effective transitions it must predict a shifted agent position. The predictor implicitly learns this distinction, clustering no-op latents near an "identity" manifold shared across all four action directions, and effective-transition latents in action-specific regions.

Rollout Stability. Per-step PSNR plateaus near 31.9 dB after step 8 and never falls below the Stage 0 proxy (Figure 2), a behavior we attribute to $\lambda\mathcal{L}_{\text{embed}}$ creating a basin of attraction that prevents runaway drift. Without this term, Drift_5 increases 65%. The dec_partial ablation achieves 49.6 dB by jointly adapting the decoder during Stage 2 — a degenerate result since freed output layers can memorize the rendering function rather than learning a general world model; freezing the decoder is therefore an intentional inductive bias, not a limitation.

Staged Training as Optimization Decomposition. End-to-end training from scratch fails catastrophically (9.45 dB without Stage 0), because g_ϕ receives incoherent gradients before it has learned the visual domain. Each stage reduces the search space: Stage 0 establishes the visual prior, Stage 1 aligns the latent channel with the encoder manifold (cosine similarity >0.997), and Stage 2 grounds pixel-level predictions within the stabilized latent channel. This decomposition is the primary reason a 5.8 M parameter model can train successfully from scratch on a single GPU in under 3 hours.

Why WM Look-Ahead Did Not Help Planning. The tie at 5% SR between VLM-only and VLM+WM traces to two compounding issues. First, *task mismatch*: the VLM was fine-tuned to output action words but is implicitly asked to judge future states — the lookahead filter can only discard actions whose predicted frame elicits an explicit reversal, a rare signal. Second, *greedy myopia*: single-step look-ahead is equivalent to greedy search in the WM's latent space, which can still walk into dead ends without backtracking. Meaningful gains would require multi-step WM rollouts paired with a dedicated value function, or a scoring function trained directly on (predicted frame, goal) pairs.

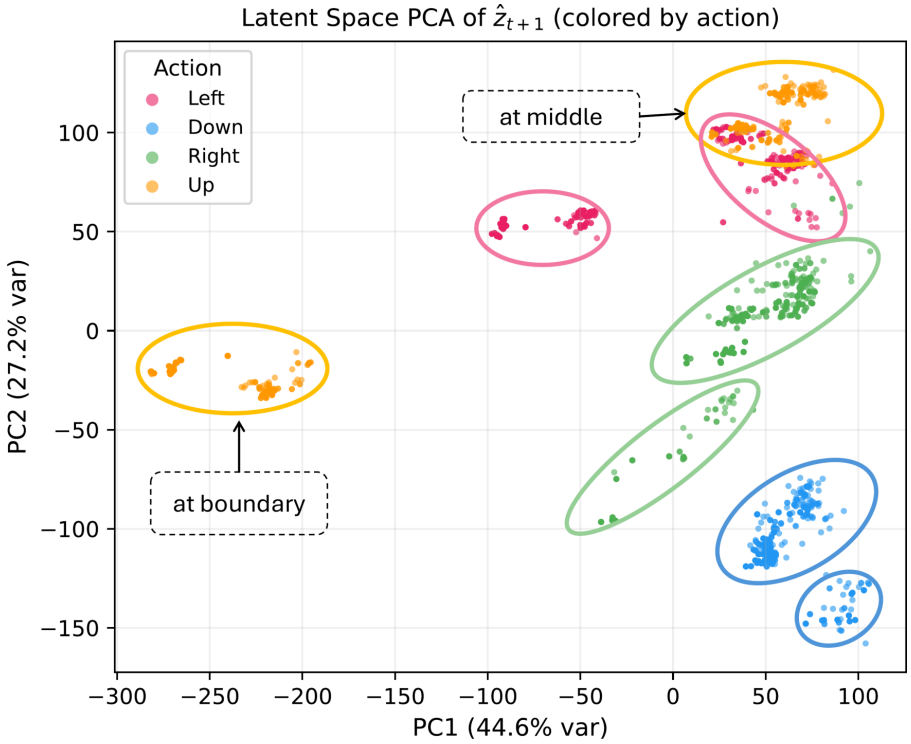

Figure 4: PCA of pooled $\hat{z}_{t+1}$ tokens colored by action (2000 test samples). Distinct clusters per action direction confirm action-conditioned latent structure. Each cluster further splits into two sub-clouds corresponding to boundary no-ops (agent at wall) and effective transitions (agent moves).

8 Conclusion

We presented a three-stage training pipeline for learning action-conditioned world models via structured latent token blocks. On FrozenLake, our model achieves 34.66 dB PSNR one-step and $\text{Drift}_{10} = 0.02341$ over 10-step rollouts, outperforming all ablation baselines. A downstream evaluation pairs the WM with a fine-tuned Qwen2-VL-2B; both VLM-only and VLM+WM reach 5% success rate, revealing that high-fidelity prediction alone is insufficient for task improvement without stronger search or a learned value function. The key insight is that separating decoder adaptation (Stage 0), latent format stabilization (Stage 1), and transition grounding (Stage 2) substantially eases optimization compared to end-to-end training from scratch.

Limitations. Compounding error causes quality degradation beyond 15 steps. The planning evaluation exposes a gap between prediction quality and task utility: single-step WM look-ahead does not improve over the VLM-alone baseline (both 5%), since greedy one-step search cannot recover from local dead ends and the reversal filter is too weak to meaningfully redirect an already-confused policy. The FrozenLake domain is also deterministic and visually simple; richer environments (stochastic transitions, continuous actions) would stress-test the approach. Future work includes multi-step WM rollouts with learned value functions, scheduled sampling, and contrastive embedding objectives.

References

[1] D. Hafner et al., *Dream to Control: Learning Behaviors by Latent Imagination*, ICLR 2020.

[2] D. Ha and J. Schmidhuber, *World Models*, NeurIPS 2018.

[3] J. Oh et al., *Action-Conditional Video Prediction using Deep Networks in Atari Games*, NeurIPS 2015.

[4] D. Hafner et al., *Mastering Atari with Discrete World Models*, ICLR 2021.

[5] A. Hu et al., *GAIA-1: A Generative World Model for Autonomous Driving*, arXiv:2309.17080, 2023.

[6] M. Burchi and R. Timofte, *MuDreamer: Learning Predictive World Models without Reconstruction*, arXiv:2405.15083, 2024.

[7] J. Bruce et al., *Genie: Generative Interactive Environments*, ICML 2024.

[8] Z. Cheng et al., *EmbodiedEval: Evaluate Multimodal LLMs as Embodied Agents*, arXiv:2501.11858, 2025.

[9] Y. Hu et al., *EWMBench: Evaluating Scene, Motion, and Semantic Quality in Embodied World Models*, arXiv:2505.09694, 2025.

[10] M. Towers et al., *Gymnasium: A Standard Interface for Reinforcement Learning Environments*, arXiv:2407.17032, 2024.

[11] Z. Wang et al., *Image Quality Assessment: From Error Visibility to Structural Similarity*, IEEE Trans. Image Process., 2004.

[12] P. Wang et al., *Qwen2-VL: Enhancing Vision-Language Model's Perception of the World at Any Resolution*, arXiv:2409.12191, 2024.